

ESCOLA SUPERIOR DE TECNOLOGIA DE SETÚBAL

Instituto Politécnico de Setúbal

Programação Orientada a Objetos

Relatório Final

Tower Defense: Guardiões da Fortaleza

Jogo Desktop em Java com JavaFX

Grupo - PL

Hugo Barreiras (2024144528)

Elioenai Vitoreira (2024146470)

Beatriz Vieira (2024149895)

Docente: Pedro Mesquita.

Ano Letivo 2025/2026 - 2.º Semestre

Índice

1. Apresentação do Projeto.....	4
1.1 Descrição do Jogo.....	4
1.2 Objetivo	4
1.3 Regras Principais	4
1.4 Condições de Vitória e Derrota	5
1.5 Forma de Interação com o Utilizador	5
1.6 Modos de Jogo.....	5
2. Modelação do Domínio	6
2.1 Entidades Principais	6
2.2 Relações entre Entidades.....	6
3. Diagrama de Classes UML	7
3.1 Hierarquias de Herança	7
3.2 Principais Atributos e Métodos	8
4. Arquitetura da Aplicação	9
4.1 Modelo de Domínio (model)	9
4.2 Controlo da Aplicação (logic).....	9
4.3 Interface Gráfica em JavaFX (ui)	10
4.4 Estrutura de Pacotes.....	10
5. Funcionalidades Implementadas	10
5.1 Sistema de Combate	10
5.2 Modos de Jogo.....	11
5.3 Geração Procedimental	11
5.4 Sistema de Economia	11
5.5 Sistema de Classificações	11
5.6 Interface Gráfica.....	12
6. Aplicação dos Conceitos de Programação Orientada a Objetos	12
6.1 Encapsulamento	12
6.2 Herança	13
6.3 Polimorfismo.....	14
7. Tratamento de Exceções.....	15
7.1 Leitura e Escrita de Ficheiros	15
7.2 Parsing de Dados.....	15

7.3 Validações Defensivas	15
8. Testes Unitários	16
9. Dificuldades Encontradas	16
9.1 Geração Procedimental de Mapas com 2 Caminhos	16
9.2 Sistema de Combate em Tempo Real	17
9.3 Interpolação de Posição dos Inimigos	17
9.4 Balanceamento do Jogo	17
9.5 Renderização em Canvas	17
10. Conclusão	18

1. Apresentação do Projeto

1.1 Descrição do Jogo

O projeto consiste num jogo de Tower Defense intitulado "Guardiões da Fortaleza", desenvolvido em Java 17 com interface gráfica em JavaFX 21. O jogador defende o seu castelo colocando heróis (torres) num mapa em grelha. Os inimigos percorrem um ou mais caminhos predefinidos e o objetivo é eliminá-los antes que cheguem ao castelo, o que causa dano na vida (HP) do castelo.

1.2 Objetivo

Sobreviver a todas as waves (ondas) de inimigos sem que o HP do castelo chegue a zero. O jogador compra e posiciona heróis estrategicamente no mapa, que atacam automaticamente os inimigos dentro do seu alcance.

1.3 Regras Principais

Os heróis só podem ser colocados em células livres (fora do caminho dos inimigos).

Cada herói tem um custo em ouro. O jogador ganha ouro ao eliminar inimigos e ao completar waves.

Os heróis atacam automaticamente os inimigos dentro do seu alcance.

Alguns inimigos atacam os heróis, podendo destruí-los. O jogador recebe 30% do custo de volta quando um herói é destruído.

Ao vender um herói voluntariamente, o jogador recebe 50% do custo de volta.

As waves ficam progressivamente mais difíceis.

Cada inimigo que chega ao castelo causa dano: inimigos normais tiram 1 HP, o Boss tira 3 HP.

1.4 Condições de Vitória e Derrota

Vitória: Sobreviver a todas as waves do nível.

Derrota: O HP do castelo chega a 0.

1.5 Forma de Interação com o Utilizador

O jogador interage através de uma interface gráfica JavaFX composta por múltiplos ecrãs. No início, insere o seu nome e acede ao menu principal, onde escolhe entre Modo História (6 níveis com progressão) ou Modo Praticar (treino livre com dificuldade à escolha). Pode ainda consultar as instruções do jogo e a tabela de classificações.

No ecrã de jogo, a interação é feita por cliques no rato: o jogador seleciona um herói no painel lateral e clica numa célula livre do mapa para o colocar. Pode selecionar uma torre existente para ver o seu alcance ou vendê-la. As waves são iniciadas manualmente pelo jogador através de um botão dedicado.

1.6 Modos de Jogo

Modo História - O jogador progride por 6 níveis com dificuldade crescente. A cada nível, novos heróis e inimigos são desbloqueados. O jogador completa um nível para desbloquear o seguinte. O Nível 6 inclui um modo endless (infinito) após as 15 waves base, onde a dificuldade continua a crescer.

Modo Praticar - O jogador escolhe entre 3 níveis de dificuldade (Fácil, Médio, Difícil) e treina livremente com todos os heróis e inimigos da dificuldade escolhida disponíveis desde o início.

2. Modelação do Domínio

2.1 Entidades Principais

Entidade	Responsabilidade
Jogo	Controla o ciclo de jogo, waves, ataques, economia e estados de vitória/derrota.
ModoJogo (abstrato)	Define os parâmetros comuns dos modos: mapa, caminhos, dificuldade, ouro e waves.
ModoHistoria	Implementa progressão por níveis com dificuldade predefinida e desbloqueio sequencial.
ModoPraticar	Permite jogar em modo Endless com dificuldade escolhida pelo jogador.
Nivel	Representa um nível do modo história com configuração de mapa, waves e personagens.
Jogador	Guarda nome, ouro, pontuação, HP e progresso.
Mapa	Representa a grelha, os caminhos, o castelo, os pontos de spawn e as posições livres.
Celula	Guarda o tipo de célula, a posição e a torre colocada quando aplicável.
Posicao	Coordenadas imutáveis (x, y) no mapa, com cálculo de distância euclidiana e Manhattan.
Torre (abstrata)	Base comum dos heróis, com vida, dano, alcance, custo e cooldown.
Inimigo (abstrato)	Base comum dos inimigos, com vida, velocidade, recompensa e movimento.
Wave	Agrupar inimigos por vaga e controlar a sequência de spawn.
GeradorMapa	Gera mapas com caminhos aleatórios (1 ou 2 caminhos com cruzamento).
GeradorWaves	Gera waves com crescimento gradual e composição limitada por tipo.
ScoreManager	Gere a leitura e escrita de highscores num ficheiro de texto.

2.2 Relações entre Entidades

O Jogo contém exatamente um Jogador (composição 1:1) e tem um ModoJogo ativo (associação).

ModoHistoria e ModoPraticar herdam de ModoJogo (herança).

ModoPraticar usa uma Dificuldade (associação com enum).

ModoHistoria contém 6 Nivel (composição 1:*).

O Mapa é composto por múltiplas Celula (composição 1:*).

Cada Celula pode ter no máximo uma Torre (agregação 0..1) e tem um TipoCelula.

Cada Wave contém múltiplos Inimigo (composição 1:*).

Torre tem 6 subclasses (herança simples).

Inimigo tem 5 subclasses diretas + InimigoBoss herda de InimigoTanque (herança multi-nível).

3. Diagrama de Classes UML

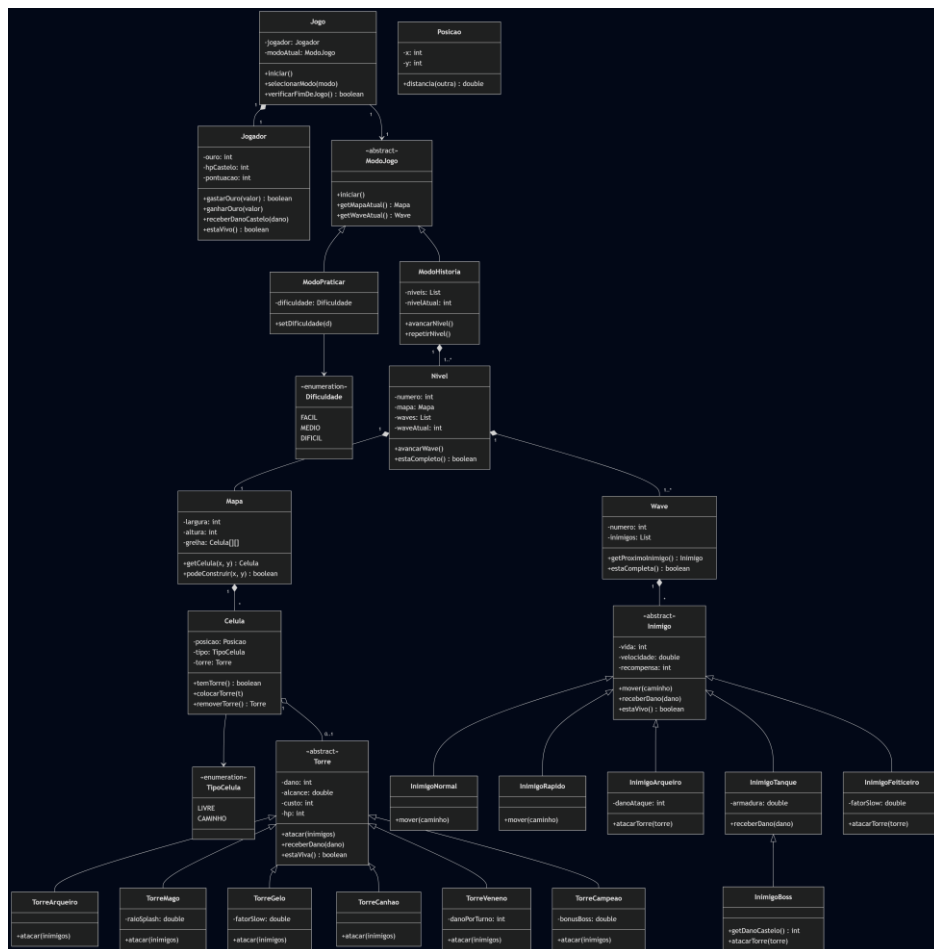


Figura 1 - Diagrama UML do projeto.

3.1 Hierarquias de Herança

Hierarquia 1 - Heróis (Torres)

Torre (abstract) |— TorreArqueiro - Ataque rápido a alvo único |— TorreMago - Dano em área (splash) até 3 inimigos |— TorreGelo - Ablanda até 3 inimigos |— TorreCanhao - Explosão que prioriza blindados |— TorreVeneno - Aplica veneno (dano contínuo) |— TorreCampeao - Dano 2x contra Boss, máx. 2 no mapa

Hierarquia 2 - Inimigos (com herança multi-nível)

Inimigo (abstract) |— InimigoNormal - Sem habilidade especial |— InimigoRapido - Velocidade alta |— InimigoArqueiro - Ataca heróis enquanto anda |— InimigoTanque - Armadura que reduz dano | |— InimigoBoss - Armadura 30%, ataca heróis, 3 HP ao castelo |— InimigoFeiticeiro - Abranda velocidade de ataque dos heróis

Hierarquia 3 - Modos de Jogo

ModoJogo (abstract) |— ModoHistoria - 6 níveis com progressão |— ModoPraticar - Jogo livre com dificuldade à escolha

3.2 Principais Atributos e Métodos

Classe Torre (abstrata)

Atributo	Tipo	Descrição
nome	String	Nome do herói
dano	int	Dano por ataque
alcance	int	Raio de alcance em células
custo	int	Custo em ouro
hp / maxHp	int	Pontos de vida atuais e máximos
velocidadeAtaque	double	Ataques por segundo
posicao	Posicao	Posição no mapa

Método	Descrição
atacar(List<Inimigo>)	Abstrato - cada torre implementa ataque de forma diferente (polimorfismo)
podeAtacar()	Verifica se o cooldown permite atacar
receberDano(int)	A torre recebe dano de inimigos
getValorVenda()	Retorna 50% do custo (venda voluntária)
getValorReembolso()	Retorna 30% do custo (destruição)
inimigosNoAlcance(List<Inimigo>)	Filtra inimigos dentro do alcance usando posição interpolada

Classe Inimigo (abstrata)

Atributo	Tipo	Descrição
vida / maxVida	int	Pontos de vida
velocidadeBase	double	Velocidade de movimento
recompensa	int	Ouro concedido ao morrer
caminho	List<Posicao>	Caminho a seguir no mapa
abrandado / envenenado	boolean	Efeitos de estado ativos

Método	Descrição
habilidadeEspecial(List<Torre>)	Abstrato - comportamento único por tipo (polimorfismo)
mover(double)	Move o inimigo ao longo do caminho com interpolação
receberDano(int)	Recebe dano, aplicando armadura via calcularDanoEfetivo()
calcularDanoEfetivo(int)	Override no InimigoTanque para redução de armadura
getDanoCastelo()	Retorna 1 por defeito, override no Boss para retornar 3
aplicarAbrandamento(double, double)	Aplica efeito de abrandamento com duração
aplicarVeneno(int, double)	Aplica efeito de veneno com dano por tick e duração

4. Arquitetura da Aplicação

A aplicação está organizada em **3 camadas** bem definidas, seguindo o princípio de separação de responsabilidades:

4.1 Modelo de Domínio (model)

O pacote `pt.poo.towerdefense.model` contém todas as entidades do domínio do jogo, sem qualquer dependência da interface gráfica. Inclui as classes base (`Posicao`, `Celula`, `Mapa`, `Wave`, `Jogador`, `Nivel`), os modos de jogo (`ModoJogo`, `ModoHistoria`, `ModoPraticar`), as hierarquias de heróis e inimigos (sub-pacotes `torre` e `inimigo`) e as enumerações (`TipoCelula`, `Dificuldade`).

4.2 Controlo da Aplicação (logic)

O pacote `pt.poo.towerdefense.logic` contém a lógica de controlo do jogo, separada do modelo e da interface. A classe `Jogo` é o controlador principal que gere o ciclo do jogo: inicializa partidas, processa cada frame (`mover inimigos` → `torres atacam` → `inimigos usam habilidades` → `verificar destruição` → `verificar fim de wave`) e usa callbacks (`Runnable`) para notificar a UI de eventos sem depender dela diretamente. As classes `GeradorMapa` e `GeradorWaves` tratam da geração procedimental de mapas e waves. A classe `ScoreManager` gere a persistência de highscores em ficheiro.

4.3 Interface Gráfica em JavaFX (ui)

O pacote `pt.poo.towerdefense.ui` contém toda a interface gráfica, construída programaticamente em JavaFX (sem FXML). É composto por 7 ecrãs: `EcranNome` (inserção do nome), `MenuPrincipal` (menu com 5 opções), `SelecaoNivel` (grelha 2×3 de níveis), `SelecaoDificuldade` (3 cards coloridos), `EcranJogo` (ecrã principal com Canvas, painel de heróis e barra de estado), `EcranInstrucoes` (TabPane com regras, heróis e inimigos) e `EcranHighscores` (tabela de classificações).

A classe principal `App` estende `javafx.application.Application` e gere a navegação entre ecrãs. O estilo visual é definido num ficheiro CSS externo (`style.css`).

4.4 Estrutura de Pacotes

`pt.poo.towerdefense` |— `App.java` (ponto de entrada JavaFX) |— `model/` (domínio) | |— `Celula`, `Mapa`, `Posicao`, `Jogador`, `Nivel`, `Wave` | |— `ModoJogo`, `ModoHistoria`, `ModoPraticar` | |— `enums/` → `TipoCelula`, `Dificuldade` | |— `torre/` → `Torre` + 6 subclasses | |— `inimigo/` → `Inimigo` + 6 subclasses |— `logic/` (controlo) | |— `Jogo`, `GeradorMapa`, `GeradorWaves`, `ScoreManager` |— `ui/` (interface gráfica) |— `EcranNome`, `MenuPrincipal` |— `SelecaoNivel`, `SelecaoDificuldade` |— `EcranJogo`, `EcranInstrucoes`, `EcranHighscores`

5. Funcionalidades Implementadas

5.1 Sistema de Combate

O jogo implementa um sistema de combate completo com **6 tipos de heróis**, cada um com mecânica de ataque distinta:

Herói	Dano	Alcance	Custo	HP	Mecânica
Arqueiro	20	3	65	100	Alvo único rápido.
Mago	40	4	115	80	Dano em área até 3 inimigos.
Torre de Gelo	8	3	100	90	Abranda inimigos próximos.
Canhão	160	2	140	120	Prioriza inimigos com mais vida, dano 1.5× contra blindados
Torre Veneno	20	3	120	100	Aplica veneno (10 dano/s durante 3s), prioriza não-envenenados
Campeão	120	3	250	200	Dano 2× contra Boss, prioriza Boss, máx. 2 no mapa

E 6 tipos de inimigos com habilidades únicas:

Inimigo	Vida	Velocidade	Recompensa	Habilidade
Normal	130	1.0	5g	Sem habilidade especial.
Rápido	80	2.0	7g	Velocidade alta
Arqueiro Inimigo	100	0.8	10g	Ataca a torre mais próxima no alcance
Tanque	400	0.5	20g	Vida/armadura altas.
Feiticeiro	90	0.7	14g	Cura aliados próximos.
Boss	1200	0.3	70g	Reduz temporariamente dano das torres.

5.2 Modos de Jogo

Modo História - 6 níveis com progressão e desbloqueio progressivo de personagens.

Modo Praticar - 3 níveis de dificuldade (Fácil, Médio, Difícil).

Modo Endless - No Nível 6, após as 15 waves base, o jogo continua infinitamente com multiplicador de vida crescente nos inimigos.

5.3 Geração Procedimental

Mapas aleatórios - Cada partida gera um mapa diferente com caminhos em serpentina. Com 2 caminhos, o sistema valida cruzamento e espaçamento mínimo, com fallback estático garantido.

Composição dinâmica de waves - As waves escalam gradualmente em quantidade e variedade conforme o progresso do nível.

5.4 Sistema de Economia

Ouro inicial configurável por nível/dificuldade.

Recompensas por inimigo eliminado e bônus por wave completada.

Venda de torres (50% do custo) e reembolso por destruição (30%).

Bônus de pontuação por completar wave sem perder HP do castelo.

5.5 Sistema de Classificações

Persistência de highscores em ficheiro de texto (scores.txt).

Top 10 ordenado por waves sobrevividas e depois pontuação.

Cálculo de pontuação final: recompensas + HP restante $\times$ 50 + ouro restante.

Ranking guardado em ficheiro e limitado ao Top 10.

5.6 Interface Gráfica

Ecrãs com animações de entrada (fade, slide).

Canvas com renderização detalhada: sprites personalizados para cada tipo de torre e inimigo desenhados com primitivas gráficas.

Decoração procedimental do mapa (erva, pedras, flores, arbustos).

Sistema de projéteis animados com trail visual e explosões de impacto.

Barras de HP coloridas, indicadores de efeitos de estado (abrandamento, veneno).

Tooltips informativos nos botões de heróis com stats detalhados.

Ecrã de instruções com tabs organizadas (Regras, Heróis, Inimigos).

6. Aplicação dos Conceitos de Programação Orientada a Objetos

6.1 Encapsulamento

Todos os atributos das classes são declarados como `private` (ou `private final` quando imutáveis), com acesso controlado via getters e, quando necessário, setters específicos com validação.

Exemplo - Classe Jogador:

Os atributos `ouro`, `hpCastelo` e `pontuacao` são privados. A gestão do ouro é feita através dos métodos `gastarOuro(int)` e `ganharOuro(int)`. O método `gastarOuro()` valida internamente se o jogador tem ouro suficiente antes de deduzir, retornando `false` caso contrário. Isto impede alterações diretas e inconsistentes ao saldo:

```
public boolean gastarOuro(int quantidade) { if (ouro >= quantidade) { ouro -= quantidade;
return true; } return false; }
```

Exemplo - Classe Celula:

O método `colocarTorre(Torre)` valida internamente se a célula está livre antes de aceitar a torre, e define automaticamente a posição da torre. O exterior não consegue colocar uma torre numa célula ocupada:

```
public boolean colocarTorre(Torre torre) { if (!isLivre()) return false; this.torre = torre;
torre.setPosicao(posicao); return true; }
```

Exemplo - Classe Inimigo:

Os efeitos de estado (abrandamento, veneno) são geridos internamente com duração e acumulação controlada. O exterior apenas invoca `aplicarAbrandamento()` ou `aplicarVeneno()`, sem aceder diretamente às variáveis de estado internas.

6.2 Herança

O projeto implementa **3 hierarquias de herança**:

Hierarquia de Torres (herança simples com 6 subclasses):

A classe abstrata `Torre` define os atributos comuns (dano, alcance, custo, HP, cooldown de ataque) e o método abstrato `atacar(List<Inimigo>)`. Cada uma das 6 subclasses (`TorreArqueiro`, `TorreMago`, `TorreGelo`, `TorreCanhao`, `TorreVeneno`, `TorreCampeao`) herda toda a funcionalidade base e implementa apenas o comportamento de ataque específico.

Hierarquia de Inimigos (herança multi-nível):

A classe abstrata `Inimigo` define a movimentação, sistema de efeitos de estado e barra de HP. A classe `InimigoTanque` introduz o conceito de armadura com override de `calcularDanoEfetivo()`. A classe `InimigoBoss` herda de `InimigoTanque` (não diretamente de `Inimigo`), reutilizando a mecânica de armadura e adicionando as suas próprias habilidades. Esta cadeia `Inimigo` → `InimigoTanque` → `InimigoBoss` demonstra herança multi-nível.

Hierarquia de Modos de Jogo:

`ModoHistoria` e `ModoPraticar` herdam de `ModoJogo` (abstrato) e implementam os métodos abstratos que retornam a configuração do jogo, permitindo ao controlador `Jogo` operar sobre qualquer modo sem conhecer o tipo concreto.

6.3 Polimorfismo

O polimorfismo é amplamente utilizado em todo o projeto:

Torre.atacar(List<Inimigo>) - Cada torre implementa o ataque de forma diferente:

TorreArqueiro - ataca o inimigo mais próximo (alvo único).

TorreMago - ataca até 3 inimigos em área (splash).

TorreCanhao - prioriza inimigos com mais vida, dano aumentado contra blindados.

TorreVeneno - prioriza inimigos não envenenados para maximizar cobertura.

TorreCampeao - prioriza o Boss com dano dobrado.

Inimigo.habilidadeEspecial(List<Torre>) - Cada inimigo tem comportamento distinto:

InimigoNormal e InimigoRapido - não fazem nada (sem habilidade especial).

InimigoArqueiro - ataca a torre mais próxima no alcance com cooldown.

InimigoFeiticeiro - abranda a torre mais próxima e causa dano.

InimigoBoss - ataca a torre mais próxima com dano elevado.

Inimigo.getDanoCastelo() - Retorna 1 por defeito em Inimigo, mas InimigoBoss faz override para retornar 3. O controlador Jogo invoca inimigo.getDanoCastelo() polimorficamente sem verificar o tipo:

```
// No Jogo.atualizar(): jogador.receberDano(inimigo.getDanoCastelo()); // Funciona  
corretamente para todos os tipos de inimigo
```

Inimigo.calcularDanoEfetivo(int) - Retorna o dano sem alteração por defeito. InimigoTanque faz override para aplicar redução de armadura. InimigoBoss herda automaticamente esta redução do InimigoTanque.

7. Tratamento de Exceções

7.1 Leitura e Escrita de Ficheiros

A classe `ScoreManager` trata exceções de I/O (`IOException`) na leitura e escrita do ficheiro de `highscores` (`scores.txt`). São utilizados blocos `try-with-resources` que garantem que os ficheiros são sempre fechados corretamente, mesmo em caso de erro:

```
// Leitura de scores com try-with-resources
try (BufferedReader reader = new
    BufferedReader(new FileReader(ficheiro))) {
    String line;
    while ((line = reader.readLine()) != null) {
        // processar linha...
    }
} catch (IOException e) {
    System.err.println("Erro ao ler scores: " + e.getMessage());
}

// Escrita de scores com try-with-resources
try (BufferedWriter writer = new BufferedWriter(new FileWriter(FICHEIRO_SCORES))) {
    for (ScoreEntry score : scores) {
        writer.write(score.toFileLine());
        writer.newLine();
    }
} catch (IOException e) {
    System.err.println("Erro ao guardar scores: " + e.getMessage());
}
```

7.2 Parsing de Dados

O método `ScoreEntry.fromFileLine(String)` trata dados corrompidos no ficheiro de scores, verificando o número de campos antes de fazer parsing. Suporta o formato atual (6 campos) e o formato antigo (5 campos, marcado como "LEGACY"), retornando `null` para linhas inválidas que são então ignoradas pela leitura.

7.3 Validações Defensivas

Em diversas classes, são feitas validações antes de operar para evitar estados inconsistentes:

`Mapa.getCelula(int, int)` - Retorna `null` se as coordenadas estiverem fora dos limites da grelha, evitando `ArrayIndexOutOfBoundsException`.

`Jogador.gastarOuro(int)` - Verifica se o saldo é suficiente antes de deduzir.

`Celula.colocarTorre(Torre)` - Verifica se a célula está livre antes de aceitar.

`Inimigo.mover()` - Verifica múltiplas pré-condições (vivo, não chegou ao castelo, caminho válido).

`Jogo.colocarTorre()` - Encadeia validações: célula válida, célula livre, ouro suficiente, limite de Campeões, presença de Boss.

`GeradorMapa` - Valida espaçamento entre caminhos e limites do mapa, com fallback para layout estático caso a geração aleatória falhe após 200 tentativas.

8. Testes Unitários

A arquitetura do projeto foi desenhada de forma a facilitar a testabilidade. A separação entre as camadas model, logic e ui permite testar a lógica de jogo sem dependência da interface gráfica. A classe Jogo aceita um Random com seed fixa no construtor, permitindo reproduzir partidas exatas para testes determinísticos. O GeradorMapa e GeradorWaves também aceitam um Random externo. Na versão final foram implementados 15 testes unitários com JUnit 5, executáveis por Maven através do comando mvn test.

Os principais cenários de teste considerados incluem:

Classe / Método	Cenário de Teste
Jogador.gastarOuro()	Valida ouro insuficiente e dedução correta quando há saldo.
Jogador.receberDano()	Confirma que o HP nunca fica negativo.
Inimigo.receberDano()	Inimigo morre (isVivo() = false) quando vida chega a 0.
InimigoTanque.calcularDanoEfetivo()	Dano é reduzido pela percentagem de armadura (20%).
InimigoBoss.getDanoCastelo()	Retorna 3 (override do valor padrão 1).
Torre.getValorVenda()	Retorna exatamente 50% do custo.
Torre.getValorReembolso()	Retorna exatamente 30% do custo.
Celula.colocarTorre()	Falha em célula não-livre; sucesso em célula livre.
Posicao.equals() / hashCode()	Duas posições com mesmas coordenadas são iguais.
Wave.verificarConclusao()	Wave é concluída quando todos os inimigos morreram ou chegaram ao castelo.
ModoHistoria.completarNivelAtual()	Completar nível desbloqueia o próximo.
ScoreManager	Leitura/escrita de scores em ficheiro e ordenação correta.

9. Dificuldades Encontradas

9.1 Geração Procedimental de Mapas com 2 Caminhos

A geração aleatória de 2 caminhos que se cruzam em exatamente 1 ponto, mantendo espaçamento mínimo entre si (sem células adjacentes partilhadas), revelou-se um desafio algorítmico significativo. A solução envolveu múltiplas tentativas (até 200) com validação rigorosa de espaçamento 4-connected e sobreposição entre os caminhos. Para garantir robustez, foi implementado um sistema de fallback com layout estático testado que é utilizado caso a geração aleatória falhe.

9.2 Sistema de Combate em Tempo Real

Implementar o game loop com `AnimationTimer` do JavaFX exigiu cuidado com o cálculo preciso de `deltaTime` e a imposição de um cap máximo (0.05s) para evitar saltos temporais. O spawn escalonado de inimigos (0.35s de intervalo) foi necessário para evitar aglomeração no início da wave. A prevenção de recompensas duplicadas foi resolvida usando um `Set<Inimigo>` para rastrear inimigos já recompensados.

9.3 Interpolação de Posição dos Inimigos

Os inimigos movem-se de forma contínua entre células discretas. Implementar a interpolação visual (`getPosicaoVisual()`) para animações suaves, enquanto a lógica de alcance usa a posição interpolada para precisão, exigiu um sistema de dupla representação: `indiceCaminho` (posição discreta) e `progressoEntreCelulas` (fração entre 0.0 e 1.0).

9.4 Balanceamento do Jogo

Equilibrar os valores de dano, vida, custo e recompensa dos 12 tipos de personagens ao longo de 6 níveis com waves crescentes foi um processo iterativo. Foi necessário ajustar múltiplos parâmetros para garantir que o jogo é desafiante mas justo, especialmente no modo endless onde os inimigos recebem um multiplicador de vida crescente.

9.5 Renderização em Canvas

Desenhar sprites detalhados para cada tipo de torre e inimigo diretamente no Canvas do JavaFX (sem imagens externas) exigiu cuidado com a escala relativa ao tamanho da célula e a organização em camadas de renderização (células → decoração → torres → projéteis → inimigos → explosões). A performance foi gerida limitando projéteis simultâneos a 80 e pré-calculando a decoração do mapa.

10. Conclusão

O projeto "Guardiões da Fortaleza" implementa com sucesso um jogo Tower Defense completo e funcional em Java 17 com JavaFX 21. A aplicação é composta por mais de 30 classes Java organizadas em 3 camadas (modelo de domínio, controlo e interface gráfica), demonstrando uma arquitetura limpa e bem estruturada.

O jogo oferece uma experiência completa com 12 tipos de personagens (6 heróis e 6 inimigos), cada um com habilidades únicas e mecânicas de combate distintas. Os dois modos de jogo (História com 6 níveis de progressão e Praticar com 3 dificuldades), complementados por um modo endless no nível final, garantem rejogabilidade. A geração procedimental de mapas e waves assegura que cada partida é diferente.

A interface gráfica em JavaFX apresenta múltiplos ecrãs com animações de entrada, sprites desenhados com primitivas gráficas no Canvas, sistema de projéteis animados, decoração procedimental do mapa e tabela de classificações com persistência em ficheiro.

Os principais conceitos de Programação Orientada a Objetos foram aplicados extensivamente: encapsulamento (atributos privados com acesso controlado), herança (3 hierarquias incluindo herança multi-nível Inimigo → InimigoTanque → InimigoBoss) e polimorfismo (métodos como atacar(), habilidadeEspecial() e getDanoCastelo() com comportamento distinto por subclasse).